

Understanding Git Internals: Branching, Commits, Merging, and Collaboration

Aadya Prasad

Department of Mathematics and computing

Manipal Institute of Technology

Udupi, India

prasadaadya64@gmail.com

Abstract—This report is about what I learned while working through Git. I finished every level in both the Main tab and the Remote tab on Learn Git Branching (<https://learngitbranching.js.org>). The site walks you through commits, branches, merging, rebasing, conflicts, stashing, and how remotes actually work. After that I practised the same things on a real local repository — making commits, creating branches, dealing with a merge conflict, stashing changes, and pushing to a remote. The goal was to understand not just the commands, but what is actually happening underneath when Git stores history.

Keywords—Git, version control, branching, merging, rebasing, commits, remote repositories, Learn Git Branching

I. INTRODUCTION

Before this assignment I mostly used Git by memorising a few commands. I knew how to commit and push, but I did not really understand what a commit was or why rebasing felt different from merging. The recommended resource was Learn Git Branching, so I decided to finish both the Main and Remote tabs completely instead of just skimming a few levels. This report explains what those levels taught me and then shows how I tried the same operations on an actual repository on my machine.

II. COMPLETING LEARN GIT BRANCHING

A. Main Tab

I worked through every sequence in the Main tab. The early levels are straightforward: you make commits, create branches, and do simple merges. Things get more interesting once you reach detached HEAD and relative references (^ and ~). Those levels finally made me see that a branch is just a pointer to a commit, and that HEAD is just another pointer that can sit on a branch or float free.

Later levels cover cherry-picking, tags, and more complicated rebase situations. By the end of the Advanced Topics sequence, I was comfortable moving commits around and could picture the graph that Git builds instead of treating every command as magic.

B. Remote Tab

The Remote tab is where the site starts feeling closer to real team work. You clone a repository, watch

remote-tracking branches appear (they show up as o/main in the visualiser), and practise fetch, pull and push. The levels that force you to deal with diverged history and a locked main branch were especially useful — they show why a plain git push sometimes gets rejected and what force-pushing actually does.

After finishing both tabs, I felt I finally had a mental model of the whole system instead of a list of commands I hoped would work.

III. WHAT A COMMIT ACTUALLY IS

Git does not store diffs the way many people assume. Inside the .git/objects folder it keeps four kinds of objects. A blob is just the content of a file. A tree is like a

directory listing that points to blobs or other trees. A commit points to a tree (the snapshot of the project at that moment), to one or more parent commits, and also records the author, time and message. Tags are simply named pointers, often with extra information attached.

Every object is identified by a hash of its content. Change anything and you get a completely new hash, which is why commits are effectively permanent. History is just the chain of parent links, forming a directed acyclic graph. Once I understood that, a lot of the strange behaviour around rebase and reset started making sense.

A. Branches and HEAD

A branch is nothing more than a small file that contains a commit hash. When you check out a branch, HEAD points at that file. If you check out a raw commit hash instead, HEAD becomes “detached”.

IV. MERGING, REBASING AND CONFLICTS

Merging tries to combine two lines of work. If nothing has diverged, Git simply moves the branch pointer forward (a fast-forward). When both sides have new commits, Git creates a merge commit that has two parents.

Rebasing is different. Instead of creating a merge commit it takes your commits and replays them on top of another branch, giving them new hashes. The history stays linear, which looks cleaner, but you should never rebase commits that other people have already pulled unless everyone agrees to force-push afterwards.

V. STASHING AND FORCE PUSHES

Stashing is a simple way to put unfinished work aside. git stash saves your changes and leaves a clean working tree so you can switch branches or pull. Later you bring the changes back with git stash pop. It is one of those commands that feels minor until you need it in the middle of a messy situation.

Force pushes are more dangerous. A normal push is rejected if the remote has commits you do not have. --force (or the safer --force-with-lease) overwrites the remote tip. It is necessary after a rebase of published work, but it can erase someone else's commits if you

are not careful. The Remote tab levels that deal with diverged history make this risk very clear.

VI. PRACTICAL DEMONSTRATION

After finishing the online levels I created a real repository to try the same ideas. I initialised a repo, made a couple of commits on main, then created a feature branch, added a file and merged it back. Next, I deliberately made two branches diverge so that a merge would produce a conflict. I fixed the conflict by keeping both sides' changes, committed the merge, and checked the graph with `git log -graph`.

I also practised stashing a half-finished file, then restoring it. Finally, I set up a bare repository as a fake remote, added it with `git remote add`, and pushed main to it, including a short notes file about the Learn Git Branching levels. Everything behaved exactly the way the visualiser had prepared me for.

VII. CONCLUSION

Finishing both the Main and Remote tabs of Learn Git Branching changed how I look at Git. I no longer treat commits as mysterious snapshots or branches as heavyweight copies. They are just hashes and pointers, and once you can picture the graph most of the commands become predictable. Practising the same operations on a real repository confirmed that the visual lessons transfer directly to everyday use. I still have more to learn — submodules, interactive rebase in depth, and the newer SHA-256 object format — but the foundation feels solid now.

ACKNOWLEDGMENT

Thanks to the people who built Learn Git Branching. The visual approach made the difference between memorising commands and actually understanding them. The official Git documentation was also useful once I knew what I was looking for.

REFERENCES

- [1] Learn Git Branching - [Learn Git Branching](#)

- [2] YouTube Tutorial - [Git Tutorial for Beginners: Command-Line Fundamentals](#)

- [3] Google Docs - [About Git - GitHub Docs](#)